

Task in C# with Examples

In this article, I am going to discuss **Task in C#** with Examples. Please read our previous article where we discussed how to implement Asynchronous Programming using [Async and Await Operators in C#](#) with Examples.

Task in C#

In C#, when we have an asynchronous method, in general, we want to return one of the following data types.

1. **Task and Task<T>**
2. **ValueTask and ValueTask<T>**

We will talk about ValueTask later, Now let us keep the focus on Task. The Task data type represents an asynchronous operation. A task is basically a “promise” that the operation to be performed will not necessarily be completed immediately, but that it will be completed in the future.

What is the difference between Task and Task<T> in C#?

Although we use both of them i.e. Task and Task<T> in C# for the return data type of an asynchronous method, the difference is that the Task is for methods that do not return a value while the Task<T> is for methods that do return a value of type T where T can be of any data type, such as a string, an integer, and a class, etc. We know from basic C# that a method that does not return a value is marked with a void. This is something to avoid in asynchronous methods. So, don't use async void except for event handlers.

Example to Understand Task in C#:

In our previous example, we have written the following SomeMethod.

```
public async static void SomeMethod()
{
    Console.WriteLine("Some Method Started.....");
    await Task.Delay(TimeSpan.FromSeconds(10));
    Console.WriteLine("\nSome Method End");
}
```

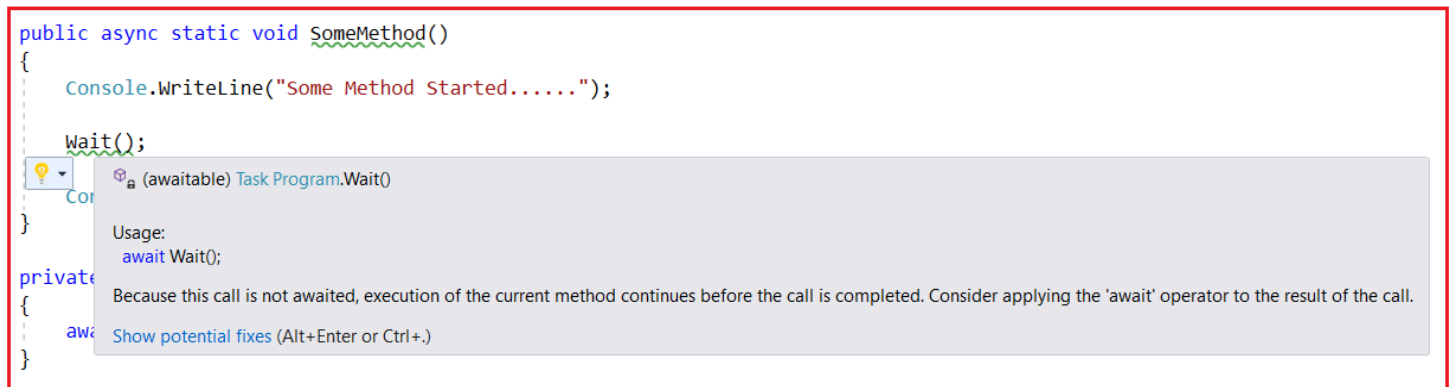
Now, what we will do is we will move the Task.Delay to separate method and call that method inside the SomeMethod. So, let's create a method with the name Wait as follows. Here, we mark the method as async so it is an asynchronous method that will not block the currently executing thread. And when calling this method it will wait for 10 seconds. And more importantly, here we use the return type as Task as this method is not going to return anything.

```
private static async Task Wait()
{
    await Task.Delay(TimeSpan.FromSeconds(10));
    Console.WriteLine("\n10 Seconds wait Completed\n");
}
```

In asynchronous programming when your method does not return anything, then instead of using void you can use Task. Now, from the SomeMethod we need to call the Wait method. If we call the Wait method like the below then we will get a warning.

```
public async static void SomeMethod()
{
    Console.WriteLine("Some Method Started.....");
    Wait();
    Console.WriteLine("Some Method End");
}
```

Here, you can see green lines under the Wait method as shown in the below image.



Why is that?

This is because the Wait method returns a Task and because it does return a Task, then it means that this will be a promise. So, this warning of the Wait method informed us that, if we don't use the await operator while calling the Wait method, the Wait method is not going to wait for this operation to finish, which means that once we call the Wait method, the next line of code inside the SomeMethod is going to be executed immediately. Let us see that practically. The following is the complete example code.

```
using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            SomeMethod();
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public async static void SomeMethod()
        {
            Console.WriteLine("Some Method Started.....");
            Wait();
            Console.WriteLine("Some Method End");
        }
    }
}
```

```

    }
    private static async Task Wait()
    {
        await Task.Delay(TimeSpan.FromSeconds(10));
        Console.WriteLine("\n10 Seconds wait Completed\n");
    }
}

```

Output: Once you execute the above code, then you will observe that without any delay we are getting the output as shown in the below image. This is because we are not using the await operator while calling the Wait method and hence it will not wait for the Wait method to complete. After 10 seconds the print statement within the Wait method is printed.

```

Main Method Started.....
Some Method Started.....
Some Method End
Main Method End

10 Seconds wait Completed

```

In the above example, we use await Task.Delay inside the Wait method. This will suspend the thread for this Wait method execution only. It will not suspend the thread for SomeMethod execution. Now, let's see what happens when we use the await operator as shown in the below example.

```

public async static void SomeMethod()
{
    Console.WriteLine("Some Method Started.....");
    await Wait();
    Console.WriteLine("Some Method End");
}

```

First thing, once you put the await operator, as shown above, the green warning will be gone. With await operator we are saying, please wait for this Wait method execution to finish before executing the next line of code inside the SomeMethod. That means it will not execute the last print statement inside the SomeMethod until the Wait method completes its execution.

```

public async static void SomeMethod()
{
    Console.WriteLine("Some Method Started.....");
    await Wait();
    Console.WriteLine("\nSome Method End");
}

```

Wait for this operation to finish before running the next line of code

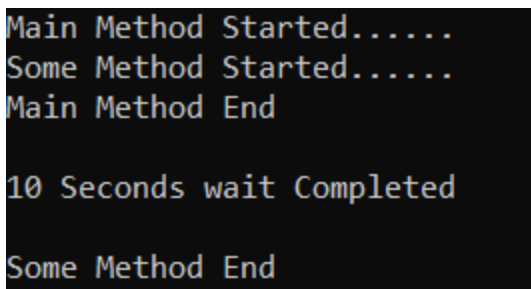
So, let us see that. The complete example code is given below

```

using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            SomeMethod();
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public async static void SomeMethod()
        {
            Console.WriteLine("Some Method Started.....");
            await Wait();
            Console.WriteLine("Some Method End");
        }
        private static async Task Wait()
        {
            await Task.Delay(TimeSpan.FromSeconds(10));
            Console.WriteLine("\n10 Seconds wait Completed\n");
        }
    }
}

```

Output:



```

Main Method Started.....
Some Method Started.....
Main Method End

10 Seconds wait Completed

Some Method End

```

Now, you can observe in the above output that once it calls the Wait method, then the SomeMethod will wait for the Wait method to complete its execution. You can see that before printing the last print statement of the SomeMethod, it prints the printing statement of the Wait method. Hence, it proves that when we use await operator then the current method execution waits until the called async method completes its execution. Once the async method, in our example Wait method, complete its example, then the calling method, in our example SomeMethod, will continue its execution i.e. it will execute the statement which is present after the async method call.

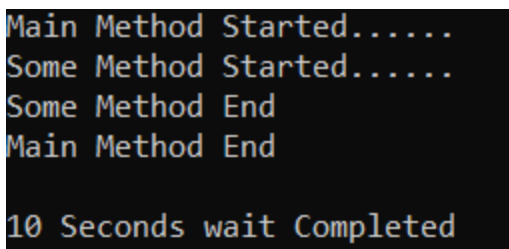
What if you don't want to wait for an asynchronous method in C#?

If you don't want your method execution to wait for the asynchronous method to complete its execution, then, in that case, you need to use the return type of the asynchronous method to void. For a better understanding, please have a look at the below example. In the below example, we have used void as the return type of the

asynchronous Wait method and while calling the asynchronous Wait method inside the SomeMethod we are not using await operator. This time please observe we are not getting any warning.

```
using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            SomeMethod();
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public async static void SomeMethod()
        {
            Console.WriteLine("Some Method Started.....");
            Wait();
            Console.WriteLine("Some Method End");
        }
        private static async void Wait()
        {
            await Task.Delay(TimeSpan.FromSeconds(10));
            Console.WriteLine("\n10 Seconds wait Completed\n");
        }
    }
}
```

Output:



```
Main Method Started.....
Some Method Started.....
Some Method End
Main Method End

10 Seconds wait Completed
```

Now you can observe the first four statements are printed immediately without waiting for the Wait method. After 10 seconds only the last statement is printed on the console window. Now, I hope you understand when to use Task and when to use void as the return type of an asynchronous method. I hope you also understand the importance of await operator.

Here, we have seen the examples of the async method without returning any value and hence we can use either void or Task as per our requirement. But what if the async method returns a value? If the async method returns a value then we need to use Task<T> and which we will discuss in our next article.

In the next article, I am going to discuss [How to Return a Value from a Task in C#](#) with Examples. Here, in this article, I try to explain **Task in C#** with Examples. I hope you enjoy this Task C# with Examples article.